MODUL

LKS PROVINSI JAWA TIMUR







CLOUD COMPUTING

2025

Project Name: Machine Learning-Powered Image Processing

Time: 6 hours

Background

As a Cloud Engineer, one of the primary challenges is designing and building a cloud infrastructure capable of handling automated, scalable, and secure image processing. The system must be able to receive image files from various sources—such as mobile applications, IoT devices, internal enterprise systems, or third-party APIs—and manage a complex processing pipeline. This pipeline includes tasks such as validating image format and size, pre-processing operations like resizing or noise reduction, and performing intelligent analysis through AI-powered algorithms like object detection, facial recognition, or content classification.

To build an efficient and reliable solution, a modern architecture that leverages managed AWS services is essential. Amazon S3 serves as the core storage service, offering high durability, scalability, and seamless integration with other AWS services. When images are uploaded to S3, automated processing can be triggered using event-driven mechanisms to invoke AWS Lambda functions. These serverless functions are ideal for lightweight operations such as validation or metadata extraction, acting as an entry point to the broader processing workflow.

For more advanced, compute-intensive workloads—especially those involving AI or ML inference—container-based processing is highly effective. Amazon Elastic Kubernetes Service (EKS) provides a robust and scalable platform to orchestrate containers that execute various stages of the image processing pipeline. For instance, different microservices can be deployed to handle preprocessing, inference with deep learning models, and result aggregation or logging. EKS supports auto-scaling and offers high fault tolerance, making it well-suited for dynamic and demanding environments.

AI-based analysis can be further enhanced by integrating Amazon Rekognition, a powerful image and video analysis service. Rekognition delivers real-time insights through object detection, facial analysis, and content moderation capabilities—eliminating the need to build and train custom ML models from scratch.

Security remains a top priority in this architecture. Fine-grained access controls are enforced through AWS Identity and Access Management (IAM), following the principle of least privilege to ensure secure access between services. All data is encrypted both in transit and at rest, and system behavior is continuously monitored using Amazon CloudWatch to detect performance issues or anomalous activity.

By leveraging these AWS services effectively, a Cloud Engineer can deliver an image processing solution that is not only powerful and cost-efficient but also highly adaptable to evolving business needs, increasing traffic loads, and future enhancements.

Task:

1. Login to console AWS using Learner LAB Academy.
2. Setup infrastructure using Terraform, please read the infrastructure service.
3. Setup Lambda function for backend, please read the Lambda Function service.
4. Setup the API Gateway, please read the API Gateway service.
5. Setup and deploy container using Docker and EKS, please read the Container Service.
6. Setup token for the access application, please read the Token Service.
7. The application must be running using load balancer.
8. Verify the application from load balancer is running well.

Technical Details:

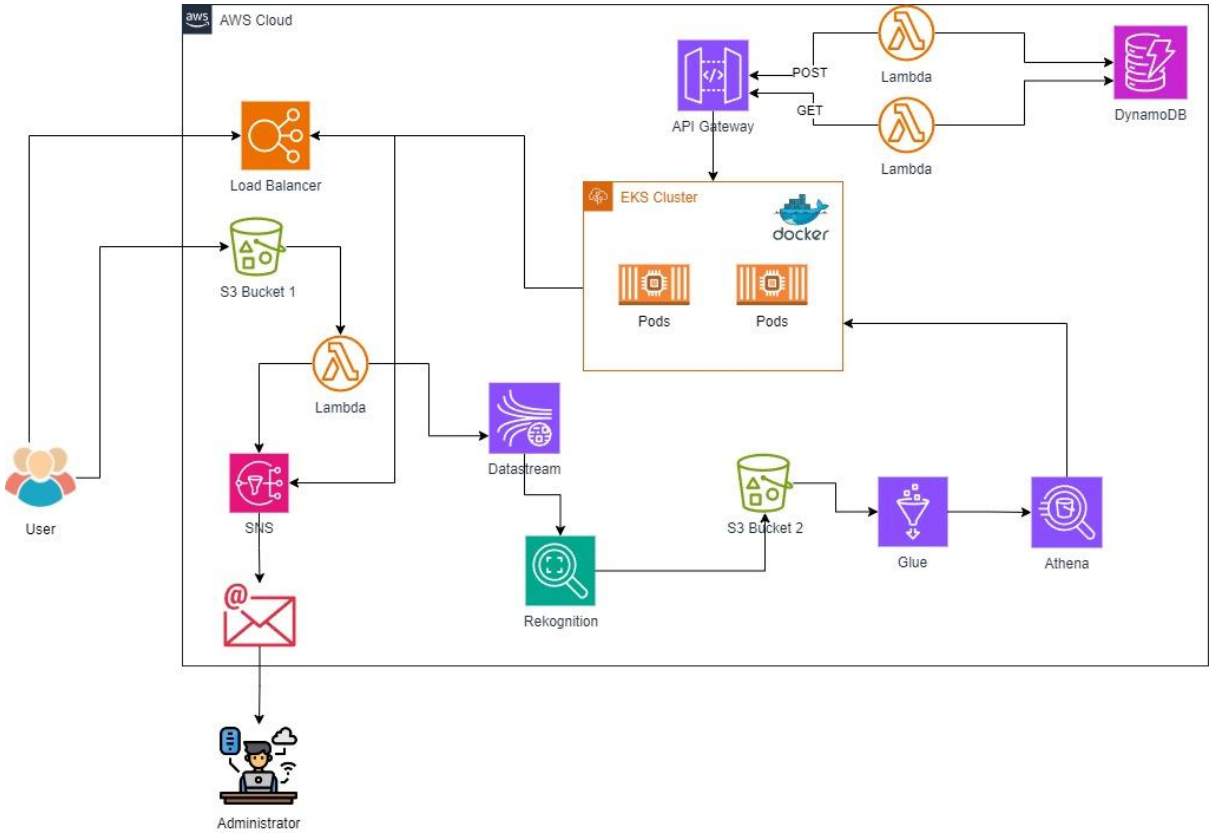
1. The goal this project is, how to the analytics image using rekognition. The apps can be access from load balancer.
2. Your default the region is us-east-1.
3. Your can access the source code from this link: https://github.com/LKS-Cloud-Computing/opsml
4. Give all the services with first name is **techno.**
5. You can use the role is LabRole.
6. If you create the manual infrastructure, you lost the point.
7. Upload the code terraform to bucket input
8. You must solve the problem in the application.
9. The source code you can access on the GitHub. Repository can be accessed on the spot.

References:

1. For documentation on S3 Bucket Policy Services you can access this link: https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html
2. For documentation API Gateway, you can access this link: https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html
3. For documentation Lambda Function, you can access on this link: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
4. For documentation EC2, you can access on this link: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html
5. For documentation Load Balancing, you can access on this link: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

6. For documentation EC2 Auto Scaling, you can access on this link: https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html
7. For documentation DockerHub, you can access on this link: https://hub.docker.com/_/python
8. For documentation Terraform, you can access on this link: https://registry.terraform.io/providers/hashicorp/aws/latest/docs
9. For documentation EKS, you can access on this link: https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html
10. For documentation Kubernetes, you can access on this link: https://kubernetes.io/docs/setup/#learning-environment
11. For documentation Rekognition, you can access on this link: https://docs.aws.amazon.com/rekognition/latest/dg/getting-started.html
12. For documentation Glue, you can access on this link: https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html
13. For documentation Athena, you can access on this link: https://docs.aws.amazon.com/athena/latest/ug/what-is.html

Architecture Diagram



- Infrastructure as Code

The user is required to create cloud computing infrastructure by setting up essential AWS services, all of which must be provisioned using Terraform as the Infrastructure as Code (IaC) tool. This includes setting up an Amazon Virtual Private Cloud (VPC) for secure networking, allowing the user to create an isolated section of the AWS cloud where resources can be launched in a defined virtual network. Additionally, the infrastructure must include Amazon S3 for scalable object storage, enabling reliable storage and retrieval of data from anywhere on the web. The code must be uploaded to an S3 bucket input.

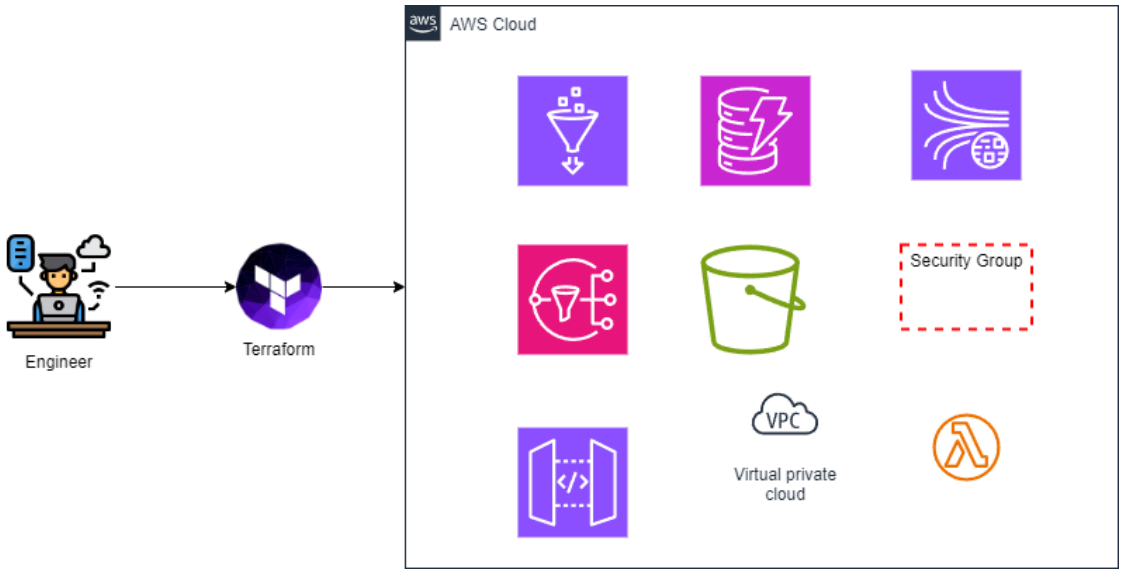
For communication between distributed systems or services, the user must configure Amazon SNS (Simple Notification Service) to enable publish-subscribe messaging patterns. To handle real-time data ingestion and processing, Amazon

Kinesis Data Streams must be provisioned, supporting the continuous capture and analysis of streaming data.

Furthermore, the infrastructure should incorporate Amazon DynamoDB as a fast and highly available NoSQL database service, offering seamless performance at any scale. Lastly, the user is required to set up AWS Glue, a serverless data integration service, to perform automated extract, transform, and load (ETL) operations, making it easier to prepare data for analytics and machine learning workflows.

In addition, the infrastructure must include **AWS Lambda**, a serverless compute service that allows code execution in response to events without provisioning or managing servers. This enables event-driven architecture and supports seamless integration with other AWS services. To expose Lambda functions as RESTful endpoints, **Amazon API Gateway** must also be provisioned. This will facilitate the creation, deployment, and management of secure APIs that connect client applications with Lambda functions or other backend services.

All these services are to be defined and deployed using Terraform, ensuring that the infrastructure is reproducible, version-controlled, and consistent across environments. If you create resources manually, **you lose the point**.



- **Virtual Private Cloud (VPC)**

  You need to configure a Virtual Private Cloud (VPC) with a CIDR block of 25.1.0.0/16 and an IPv6 range provided by Amazon, give the name is techno-yourname. The VPC should include two public subnets with IPv4 CIDR ranges 25.1.0.0/24 and 25.1.2.0/24 and IPV6 /64, as well as two private subnets with IPv4 CIDR ranges 25.1.1.0/24 and 25.1.3.0/24. Additionally, you must create two route tables: one for public subnets and one for private subnets. And you create NAT Gateway for private subnets.

- **Security Group**

  You are required to configure Security Groups to control inbound traffic for different components of the infrastructure. First, create a Security Group named **techno-sg-lb**, which will be used for the Load Balancer, and allow incoming traffic on port 80 (HTTP) and port 443 (HTTPS) to public access. The second Security Group, give the name is **techno-sg-apps,** open port 2000 to public access.

- **Simple Storage Service (S3)**

  As part of cloud infrastructure best practices, the user is required to provision **two Amazon S3 buckets** using **Terraform**, each serving a specific role in the application's data pipeline. The first bucket, named following the convention ***technoinput-yourcity-yourname***, is intended to store incoming image files uploaded from the application. The second bucket, named ***technooutput-yourcity-yourname***, is used to store processed or generated image results. Both buckets must be uniquely named and deployed in the **US East (N. Virginia) region**, ensuring global accessibility and low-latency performance.

  To support seamless integration with the application and enhance accessibility, a **bucket policy** must be applied to allow **public read access** where appropriate — particularly for assets that need to be accessed by users or external systems. This involves careful configuration of the policy JSON to avoid security misconfigurations while enabling necessary access. Users are encouraged to refer to the AWS documentation on **S3 Bucket Policies** to ensure compliance and proper implementation.

Furthermore, both S3 buckets must implement a **lifecycle policy** to manage storage efficiently. In accordance with cost optimization practices, any object stored in the buckets should be transitioned to the **Glacier storage class** (or another suitable archive tier) **after 30 days** of inactivity. Additionally, objects should be **permanently deleted after 365 days**, helping control storage costs and ensure that stale data does not accumulate indefinitely.

- **DynamoDB**

  As part of the data management strategy, the user is required to provision an **Amazon DynamoDB table** named *Tokens*, which will serve as a high-performance, scalable NoSQL database for securely storing token-related data. The table must be configured with a **partition key** named `token` of type **String**, which uniquely identifies each item in the table. This table will be the central source of truth for managing authentication or session tokens in real-time applications.

  To support real-time processing and auditing of token data, the DynamoDB table must be integrated with **Amazon Kinesis Data Stream**. This configuration allows **DynamoDB Streams** to capture data modification events (INSERT, MODIFY, REMOVE) and send them to a **Kinesis Data Stream**. With this setup, every change to the `Tokens` table will be streamed out in near real-time, enabling downstream services such as analytics engines, event processors, or machine learning pipelines to consume and act upon the data immediately.

  Activating **DynamoDB Streams with Kinesis** ensures the infrastructure adheres to modern best practices for **event-driven architecture**, allowing for loosely coupled systems and scalable real-time processing. This setup is particularly powerful in scenarios requiring traceability, anomaly detection, or monitoring access patterns across distributed systems.

- **Kinesis DataStream**

  To support real-time data ingestion and event-driven processing, the user is required to provision an **Amazon Kinesis Data Stream** named ***techno-kinesis-yourname*** using **Terraform**. This stream will act as the central pipeline for collecting and distributing time-sensitive data from various sources, such as

application logs, DynamoDB stream events, or IoT devices, enabling scalable and responsive architectures.

The stream must be configured with **Provisioned capacity mode**, which allows for fine-grained control over performance and cost. Under this mode, the user must provision **1 shard**, which provides a baseline throughput of **1 MB/sec** for write and **2 MB/sec** for read operations — sufficient for moderate to high-volume real-time data use cases. If higher throughput is needed in the future, additional shards can be added dynamically.

In alignment with data lifecycle best practices and compliance considerations, the **data retention period** should be explicitly set to **1 day (24 hours)**. This configuration ensures that streamed data is available for replay and recovery within a limited window, helping to reduce storage costs and minimize potential data lag while preserving the ability to react to transient failures or processing delays.

- **AWS Glue**

  To enable efficient **data cataloguing**, **querying**, and **ETL processing**, the user is required to provision AWS Glue resources using **Terraform**. This is an essential part of the analytics architecture, allowing structured access to semi-structured and unstructured data stored in **Amazon S3**.

  As part of this setup, a **Glue Database** named rekognition_results_db must be created. This database will serve as a **logical container** for organizing metadata tables related to image recognition outputs generated by services such as **Amazon Rekognition** or custom machine learning inference pipelines.

  Within this database, a **Glue Table** named rekognition_results_table must also be defined. This table catalogs the schema of the output data, allowing tools like **Amazon Athena** or **AWS Glue Jobs** to query and transform the data efficiently without manual schema definitions.

  The source data for this Glue table is in the S3 bucket s3://technooutput-yourcity-yourname/results
  This path is expected to contain JSON files or similar formats resulting from image analysis or machine learning processes.

In addition to the database and table, a **Glue Crawler** must be created using Terraform with the name *techno-crawler-yourname*. This crawler is responsible for **automatically detecting the schema** of the data stored in the S3 location and updating the rekognition_results_table accordingly. The crawler should be scheduled or triggered based on data arrival patterns to ensure the catalog remains up to date and queryable in near real-time.

By combining the database, table, and crawler, this setup provides a scalable and automated solution for managing metadata and enables serverless analytics on processed image data without manual intervention.

- **Simple Notifications Services (SNS)**

  As part of the alerting and messaging infrastructure, the user is required to provision an **Amazon Simple Notification Service (SNS)** topic using **Terraform**. This topic will be used to facilitate reliable and scalable messaging between AWS services and subscribers, particularly for monitoring, alerting, and event-driven notifications.

  The SNS topic must be created with a name following the standardized format: techno-sns-yourcity-yourname
  This naming convention ensures consistency across different environments and supports easier identification and traceability within a multi-team or multi-project AWS account.

  Once the topic is created, an email subscription must be configured so that all messages published to the topic are delivered to the following address: handi@seamolec.org.

- **LAMBDA FUNCTION**

  As part of the serverless and event-driven architecture, three AWS Lambda functions must be created by terraform to handle data ingestion, persistence, and retrieval processes. These functions serve distinct roles and integrate seamlessly with other AWS services, enabling a decoupled and highly responsive system.

  - **Lambda Triggered by S3 (Image Processing Trigger)**

The first Lambda function is automatically invoked when a new object (e.g., an image file) is uploaded to the **S3 bucket** technoinput-yourcity-yourname. Give the lambda function is *techno-lambda-s3* and setting the timeout is 2 minutes. This function can be used for preprocessing, validation, or sending metadata to other services like Amazon Rekognition, Kinesis Datastream, SNS. The trigger is configured via **S3 Event Notifications**, where the Lambda function is invoked on All object create events operations. This setup supports real-time reaction to user uploads and forms the entry point for the image analysis pipeline.

- **Lambda for Saving to DynamoDB (POST)**

  The second Lambda function is responsible for storing data into the **DynamoDB table** named Tokens give the lambda function with name *techno-lambda-post* and timeout 1 minute. This function typically receives a structured input (e.g., from an API Gateway or another Lambda), parses the relevant fields such as the token, and performs a PutItem operation to insert the record into DynamoDB. This Lambda must have permissions to write to the table and can be used in workflows where tokens are generated and need to be persisted securely and at scale.

- **Lambda for Retrieving from DynamoDB (GET)**

  The third Lambda function handles **data retrieval** from the Tokens table give the lambda function with name is *techno-lambda-get* and timeout 1 minute 30 seconds. It performs a GetItem or Query operation to fetch data based on the token key provided in the input event (typically a query parameter). This Lambda is ideal for use in a serverless API pattern, where frontend applications or services request the status or metadata of a token. It must be configured with read permissions to DynamoDB and optimized for low-latency response.

- **API Gateway Service**

  To configure **API Gateway**, you need to create a **REST API** named **Techno-API-yourname** with a **regional endpoint type** and use IPV4 and IPV6 for optimized performance across AWS regions. The API should have the following resources and methods:

- A **resource** named **/generate-token**, with the following HTTP methods:
  - **POST (/generate-token /POST)** to create a new token
- A **resource** named **/validate-token**, with the following HTTP methods:
  - **GET (/validate-token /GET)** to validate token.

- Container

To build and deploy a containerized application, the process begins with the creation of a **Dockerfile** for the existing project. The **Dockerfile** outlines the steps required to package the application using python3.13 and its dependencies into a portable container image. This image will ensure consistency across different environments production and facilitate scalable deployments.

Once the **Dockerfile** is complete, the next step is to push the container image to Amazon Elastic Container Registry (ECR) give the name is **techno-ecr-yourcity-yourname**. ECR provides a highly scalable and secure repository for storing container images, which can be easily integrated with other AWS services such as **Amazon ECS** or **EKS**. To achieve this, the user must authenticate Docker to the ECR registry, tag the image appropriately, and then push it into the repository for later use.

Next, the application is deployed to **Amazon Elastic Kubernetes Service (EKS)**, which provides a fully managed Kubernetes environment for running and scaling containerized applications. The **EKS cluster** should be named techno-eks-yourcity-yourname, following the specified naming convention for consistency. The cluster must be provisioned with **one node group**, which is essentially a collection of EC2 instances using t3.large that will host the Kubernetes worker nodes. These nodes are responsible for running the application containers in a scalable and fault-tolerant manner.

Within this EKS cluster, **two pods** should be deployed, each responsible for running a copy of the application container. Kubernetes pods are the smallest deployable units and ensure that your application can be scaled horizontally. By using multiple pods, the application can handle increased load and provide high availability.

For exposing the service to external traffic, a **Load Balancer** must be set up in front of the application. This allows external users to access the service running inside the

pods. The Load Balancer will automatically distribute incoming traffic across the available pods, ensuring optimal resource utilization and fault tolerance.

To deploy the service on EKS, the user must create Kubernetes YAML files—specifically **deploy.yaml** and **service.yaml**. The **deploy.yaml** file defines the Kubernetes deployment, specifying details like the number of replicas (pods), the Docker image from ECR, and any required environment variables. The **service.yaml** file defines the Kubernetes service, which acts as an internal or external access point for the pods. In this case, it will be configured to use a Load Balancer to expose the application.

After creating these files, the deployment is performed using ***kubectl apply -f***, the Kubernetes command-line tool. By applying the YAML configuration files, the EKS cluster will deploy the application, create the necessary pods, and set up the Load Balancer to handle incoming traffic. If you don't understand create file yaml, read the documentation EKS.

- Verifying Application

- **Upload the Image to S3**

  Begin by uploading an image file to the designated S3 bucket: *technoinput-yourcity-yourname*. This bucket serves as the entry point for the entire system and triggers the processing pipeline. Make sure the image file is properly formatted and uploaded to the correct path to activate the next steps.

- **Image Processed by Amazon Rekognition**

  Once the image is successfully uploaded, Amazon Rekognition is automatically triggered (via Lambda or event bridge, depending on configuration). Rekognition analyzes the image using pre-trained computer vision models to extract features such as labels, faces, or custom objects. The results are processed and stored, typically in Amazon DynamoDB or an S3 output bucket, such as *technooutput-yourcity-yourname*.

- **Access the Application via Load Balancer**

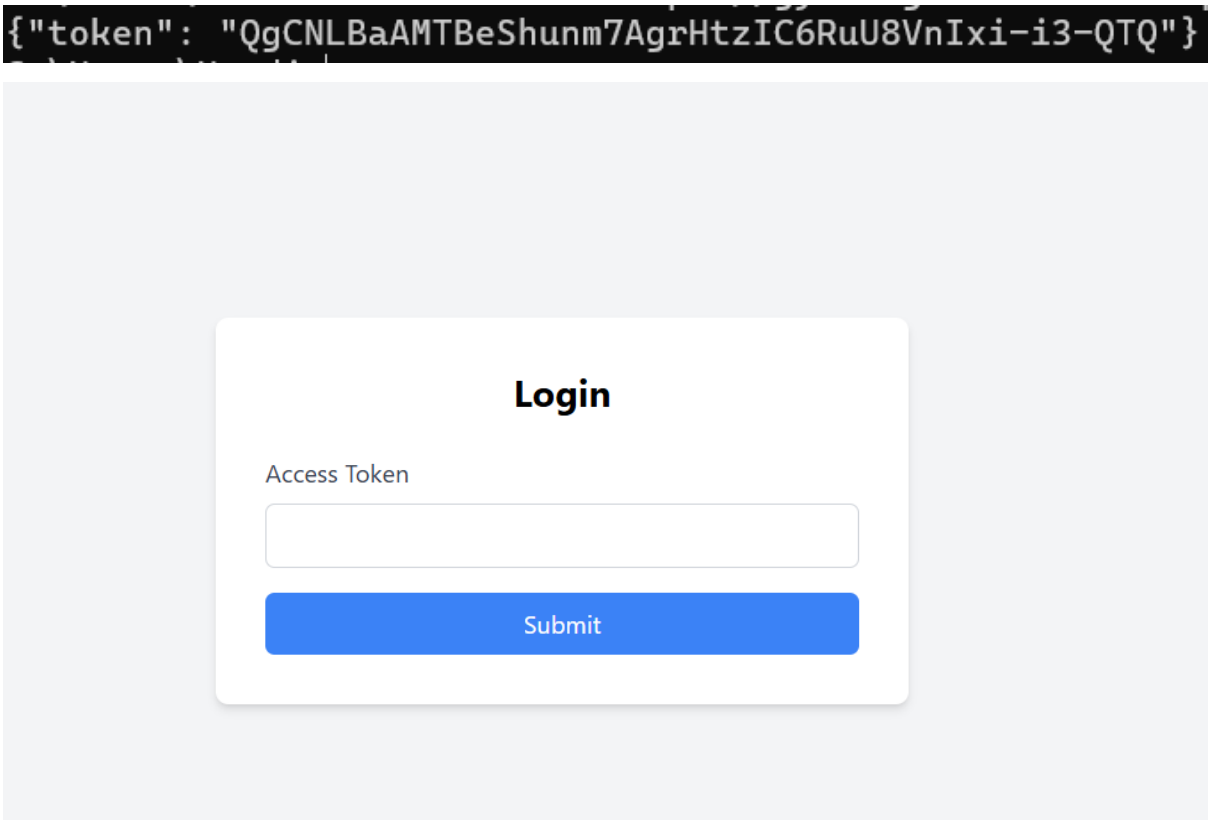
  With the image processing complete, the user should open the **Load Balancer URL** connected to the **Amazon EKS cluster**. This URL serves as the frontend access point to the containerized application running within EKS. It provides a

visual interface or API layer to interact with the system and view processed results.

- **Generate Access Token via Lambda**

  Before interacting with the machine learning application, the user must first generate a valid access token. This is done by calling the appropriate AWS Lambda function, which writes a new token to the Tokens table in DynamoDB. This token is required for authentication and access control, ensuring secure interaction with the system.

  Ex

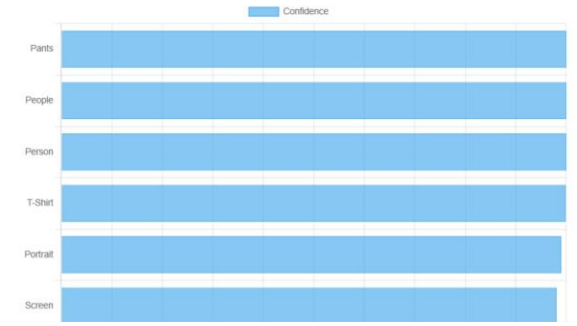


- **Machine Learning Application is Ready**

  Once a valid token is retrieved and submitted, the machine learning application becomes fully accessible, and the administrator have notification is your successfully. The user can now view the analysis results, interact with prediction services, or initiate additional image processing tasks, all within a secure and scalable cloud-native environment.

Gambar: WhatsApp Image 2025-04-04 at 14.29.43.jpeg





Gambar: WhatsApp Image 2020-12-02 at 17.29.20 (1).jpeg



